

Mikroszolgáltatások dinamikus kompozíciója az edge-cloud kontinuumban

Készítette: **Morvai Barnabás**
Neptun-kód: **E0HB0N**
Konzulens: **Czentye János**

Többen dolgoztunk ezen a témán

- Kubernetes
- Fordított programzási nyelvű kompozíció
- Bash alapú kompozíció
- Python, Lambda és Terraform...

A serverless alkalmazás futási költsége a lefoglalt memória és végrehajtási idő szorzata, a kommunikáció külön overhead

- Milliszekundum pontosságú számlázás
- Minden függvényhívás önálló számlázási egység
- Külső tárolórétegen átmenő adatáramlás extra késleltetést jelent
- Cold start inicializációs idő minden hívásnál

Az alkalmazás függvényeit külön Lambda egységekbe szervezni pazarló

- Minden függvényhívás önálló számlázási egység
- Cold start overhead minden egységnél
- Hálózati kommunikáció explicit és implicit költségei a Lambda szemszögéből
- Explicit: Ki- és beolvasás művelet elvégzésének ideje
- Implicit: A művelet elvégzése
- Implicit: Köztes állapot külső tárból való tárolása

Több önálló serverless egység egyetlen kompozit csoportba

- Függvények közötti adatáramlás kizárólag memórián belül
- Párhuzamos végrehajtás csökkenti a futási időt
- Párhuzamos végrehajtás nem igényel új serverless egységet
- Kiküszöböli a hálózati késleltetést
- Eltérő forgalmi mintákhoz, árakhoz más csoportosítás lehet optimális
- Dinamikus csoportosítással alkalmazkodni a változó körülményekhez

DEFINÍCIÓ

Atomi függvény

A végrehajtás legkisebb oszthatatlan és állapotmentes egysége. Egyetlen jól körülhatárolható üzleti logikát vagy adatfeldolgozási lépést valósít meg.

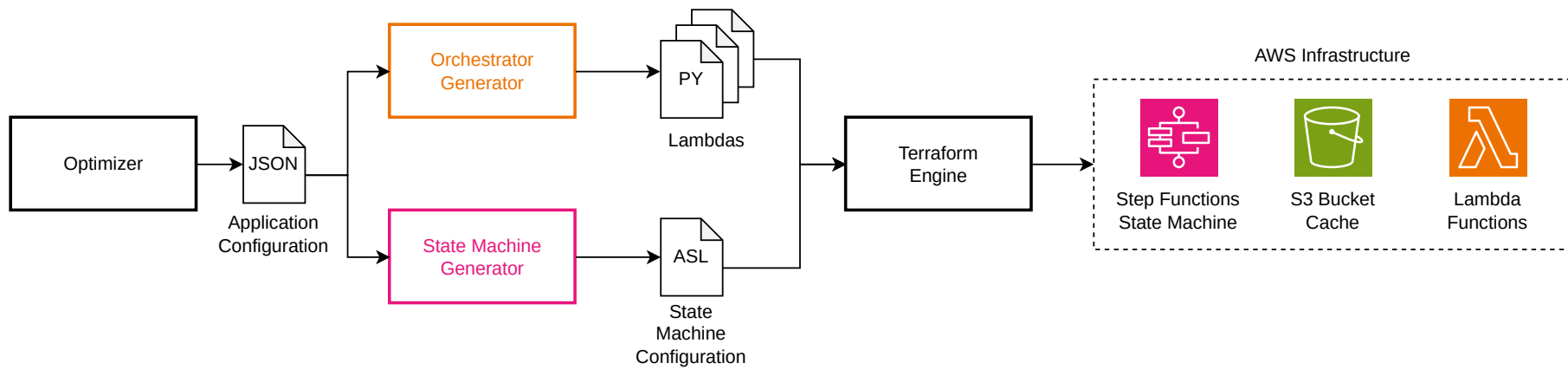
DEFINÍCIÓ

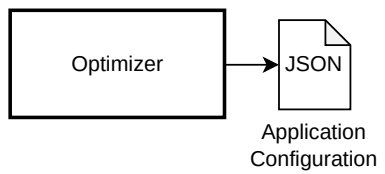
Kompozit függvény

Több atomi függvény költség- és teljesítményoptimalizált csoportosítása egyetlen, közösen futtatható AWS Lambda egységbe.

Pipeline: az optimalizáló JSON kimenete alapján automatikusan áll elő az infrastruktúra

- Az optimalizáló meghatározza az atomi függvények csoportosítását
- Orkesztrátor generátor előállítja a belépési pontokat
- Összeállítja a telepíthető Lambda függvények könyvtárait
- Előállítja a Lambda függvényeket orkesztráló állapotgépet
- Terraform deployolja az AWS infrastruktúrát
- Emberi beavatkozás nélkül összeáll és települ a rendszer AWS környezetben





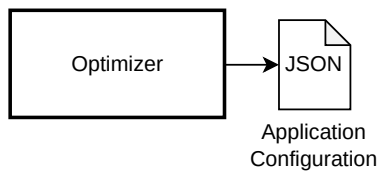
Lista írja le az egyes kompozit Lambda egységeket és a bennük szereplő atomi modulokat

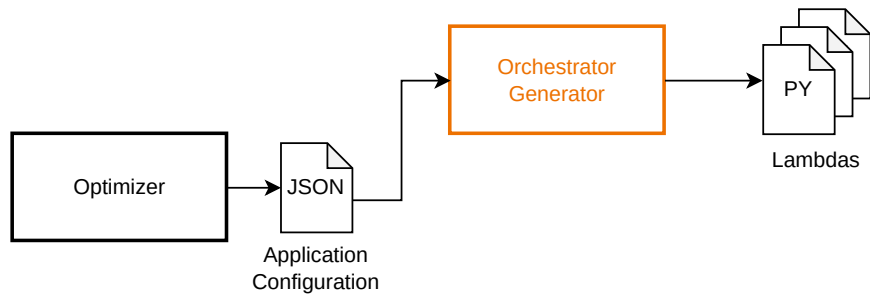
- Szándékosan lapos, egyszerű struktúra
- Minden kompozit elem egy functions listát tartalmaz
- Végrehajtási sorrend a listaelemek sorrendjéből következik
- Fa topológia nem explicit, a rendezés határozza meg az adatáramlást

HELYESSÉG

Az adatfolyam helyességét topologikus rendezés garantálja az irányított körmentes gráfon

- Minden hívási gráf DAG
- Minden DAG rendelkezik topologikus rendezéssel
- Szinkronizációs pontnál a bemeneti adatok biztosan rendelkezésre állnak
- Visszamenő él körmentes gráfban definíció szerint lehetetlen
- Atomi és kompozit esetben is működik





Az orkesztrátor vezényli az atomi függvények hívását, párhuzamosítását és állapotkezelését a kompozit függvényen belül

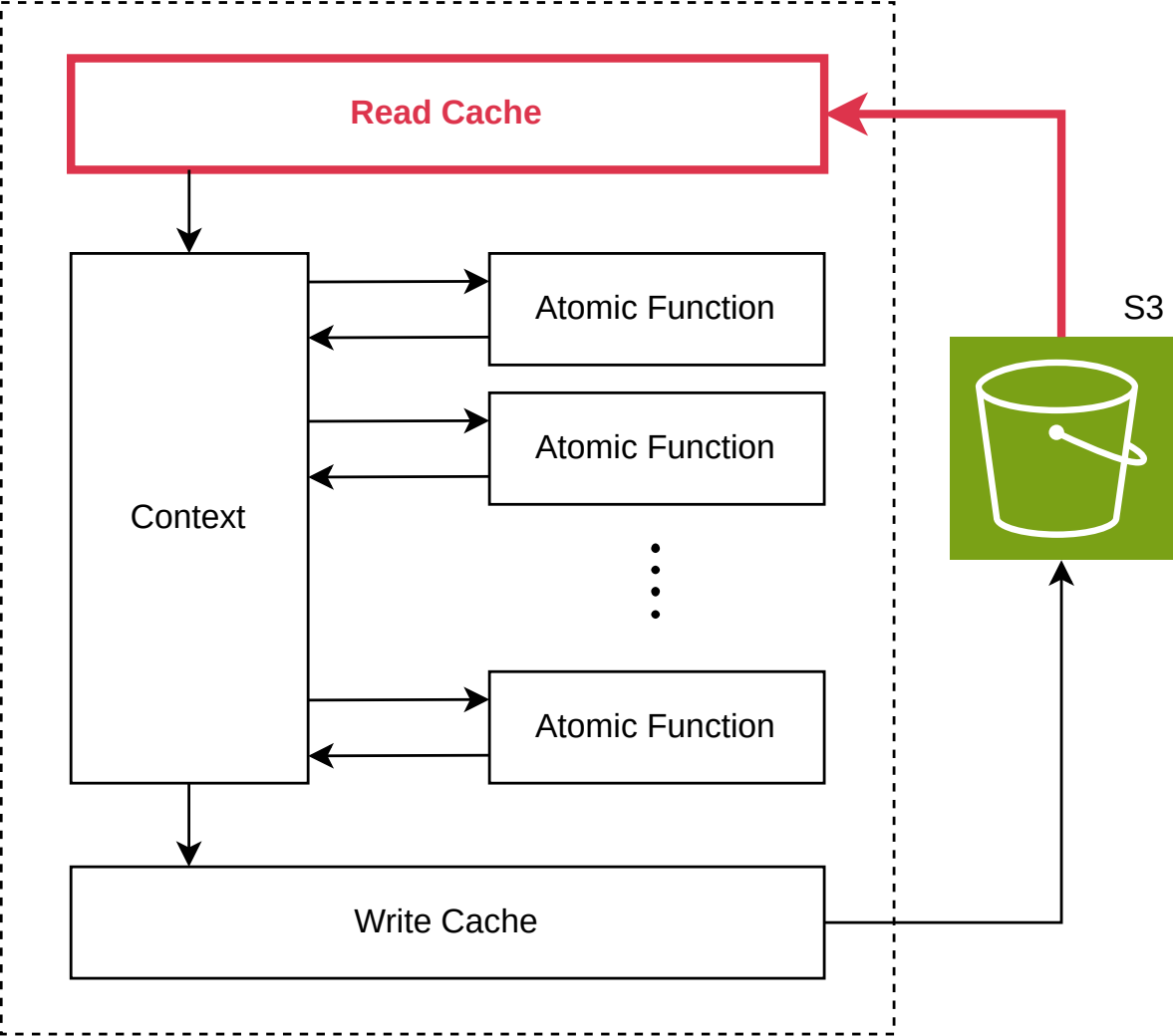
- Minden atomi függvény standard, rögzített interfésszel rendelkezik
- Interfész: bemeneti adat + context példány
- Konfigurációvezérelt, modulok konkrét implementációjától független
- Az orkesztrátor generátor ebből állítja elő a python belépési pontot
- Tartalmazza a feldolgozandó adat beolvasását
- Atomi függvények párhuzamos hívását
- Kimenetek külső tárolóba írását

CONTEXT OSZTÁLY

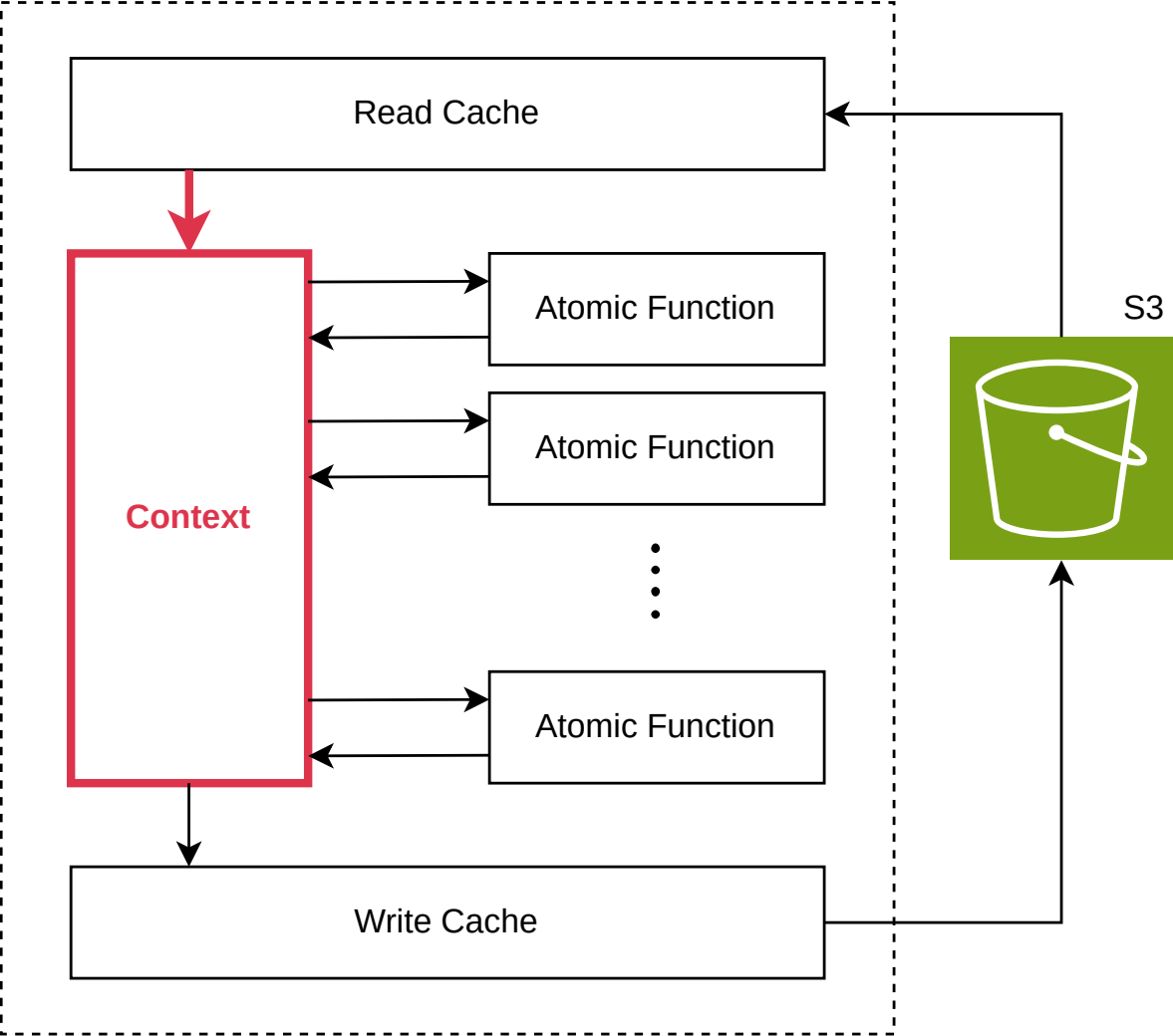
A kompozit egységen belüli adatáramlás központi eleme, amely az atomi függvények között közvetíti a köztes eredményeket

- Az atomi függvények bemenetként kapott példányba regisztrálják a kimeneteiket
- defaultdict(list) struktúra, kulcsai string típuscímkek
- Természetesen kezeli a több kimenetű eseteket
- Minden híváshoz egyedi futási azonosítóhoz kötött példány
- Destruktív olvasás: a lekérdezés eltávolítja az adatot

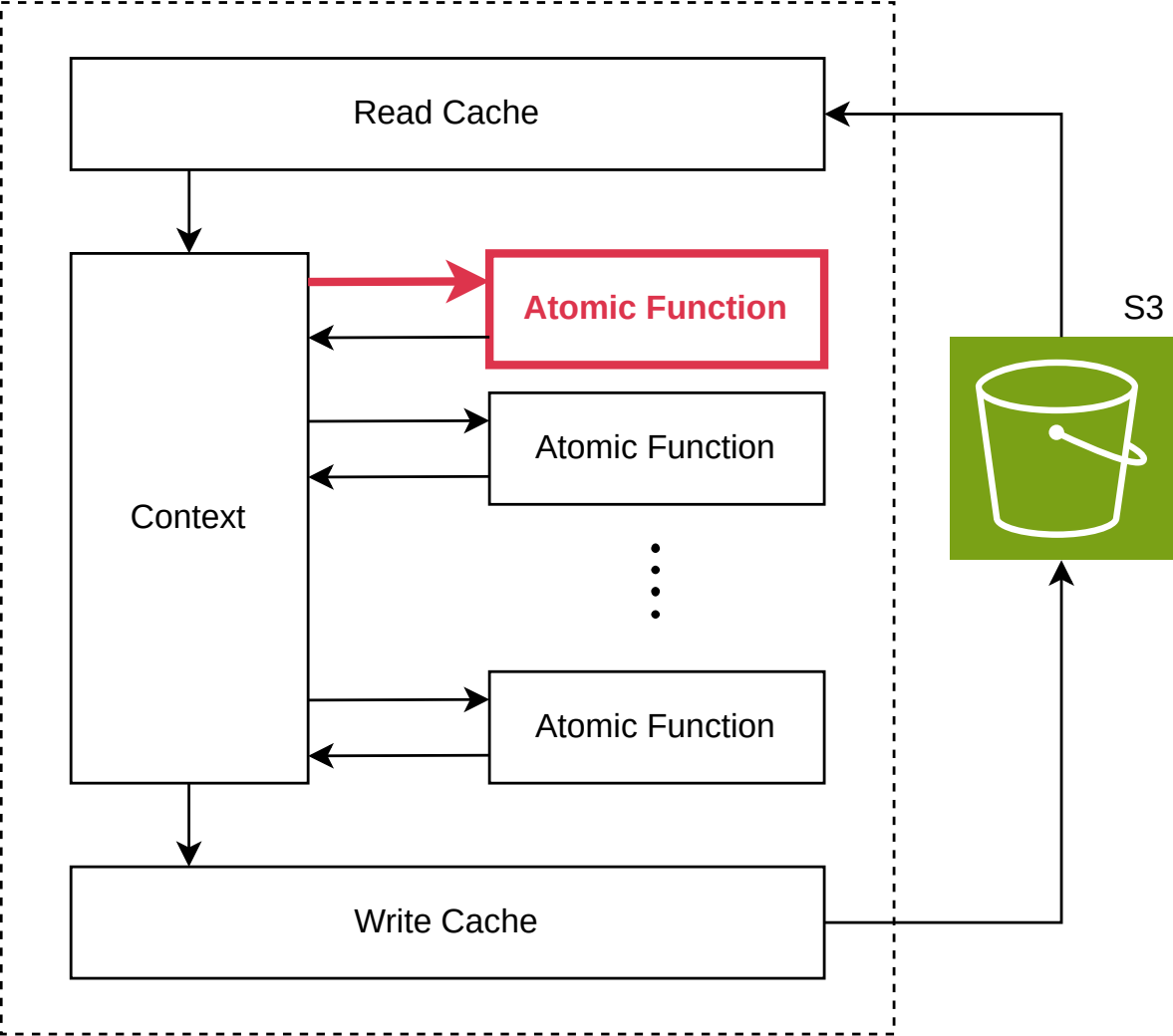
Composite Function in Lambda



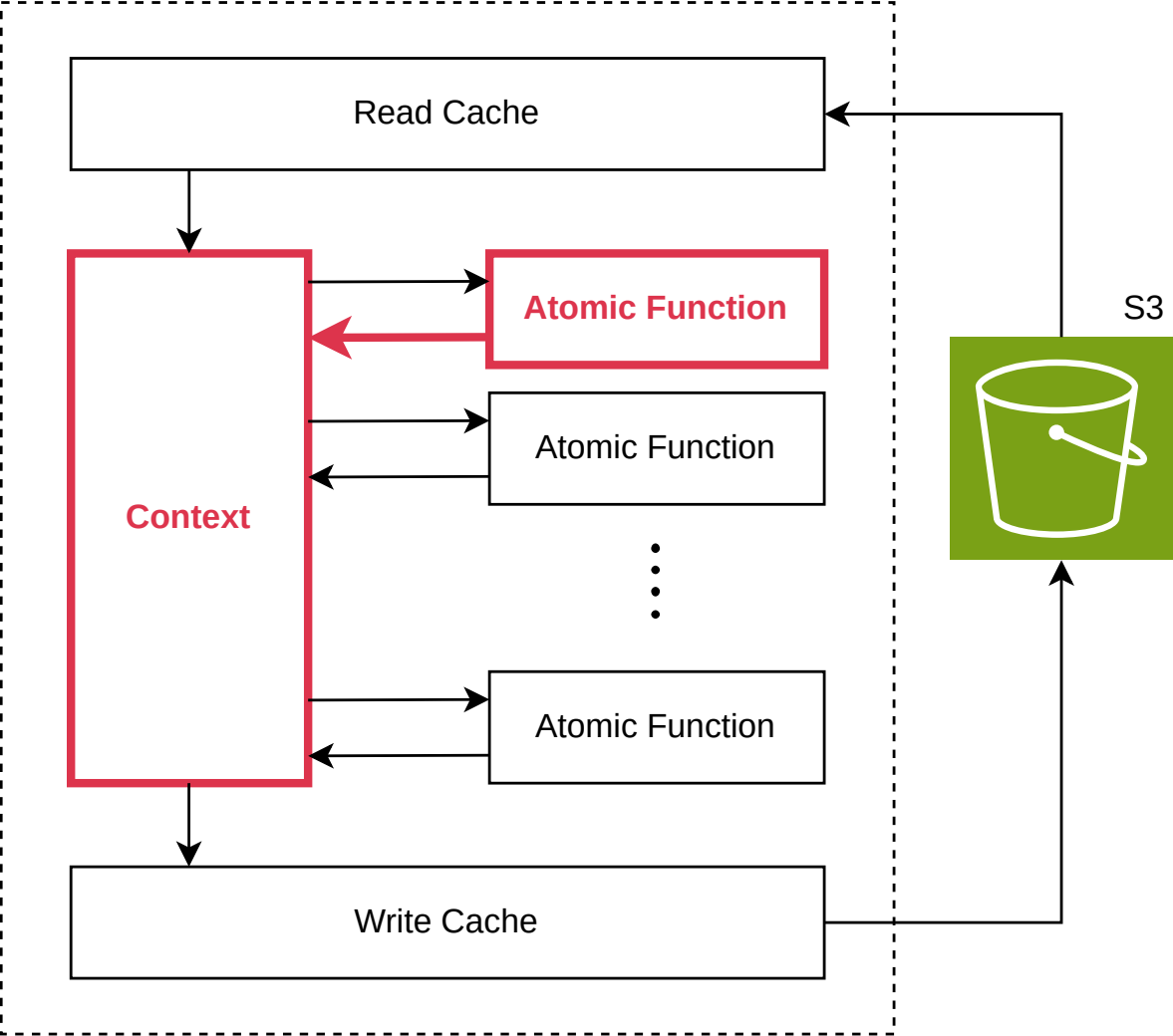
Composite Function in Lambda



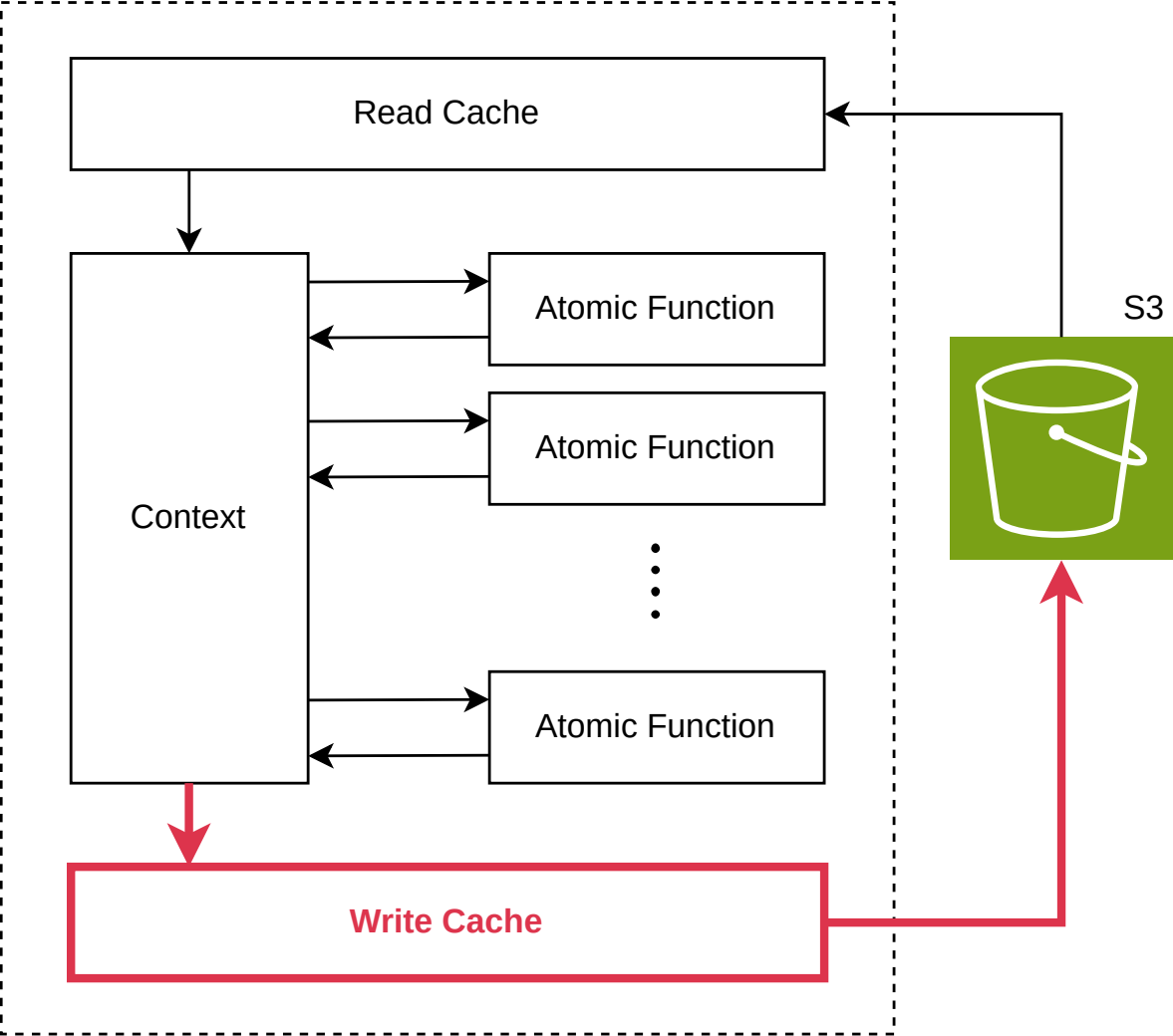
Composite Function in Lambda



Composite Function in Lambda



Composite Function in Lambda



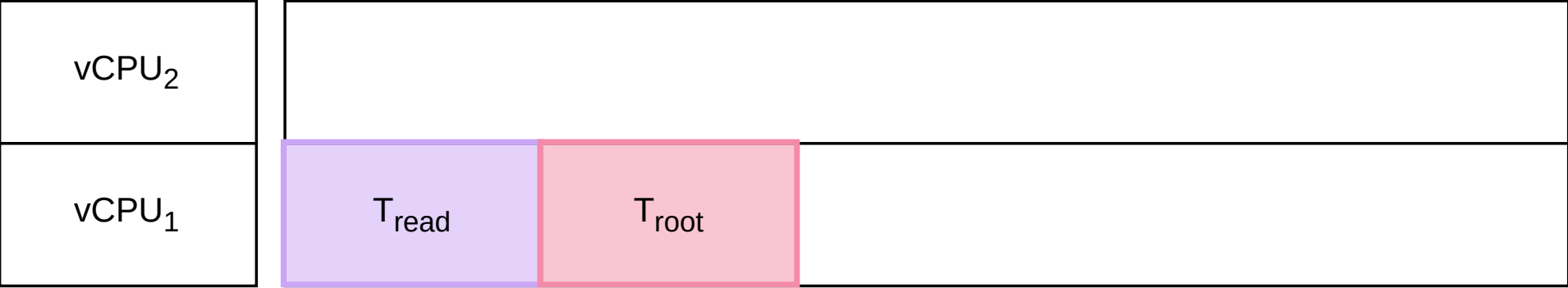
Kompozit határon az adatok S3 bucket-en keresztül, pickle szerializációval kerülnek átadásra

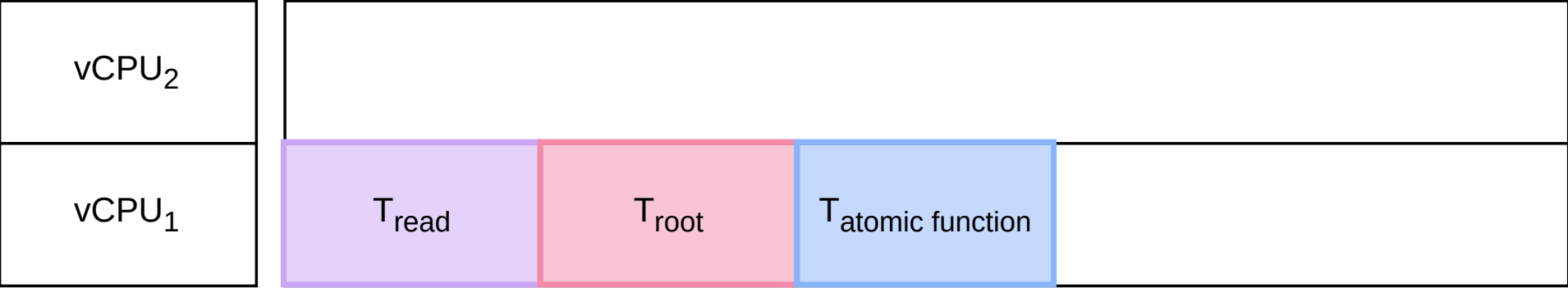
- Kiíró függvény: a context maradék kimenetei bináris formátumban S3-ba mentve
- Elérési út: stepfunctions-cache/<futási azonosító>/<típuskulcs>/
- Beolvasó függvény: bináris tartalom visszaalakulása, beregisztrálás context-be
- Az atomi függvények szempontjából nincs különbség memória és S3 forrás között

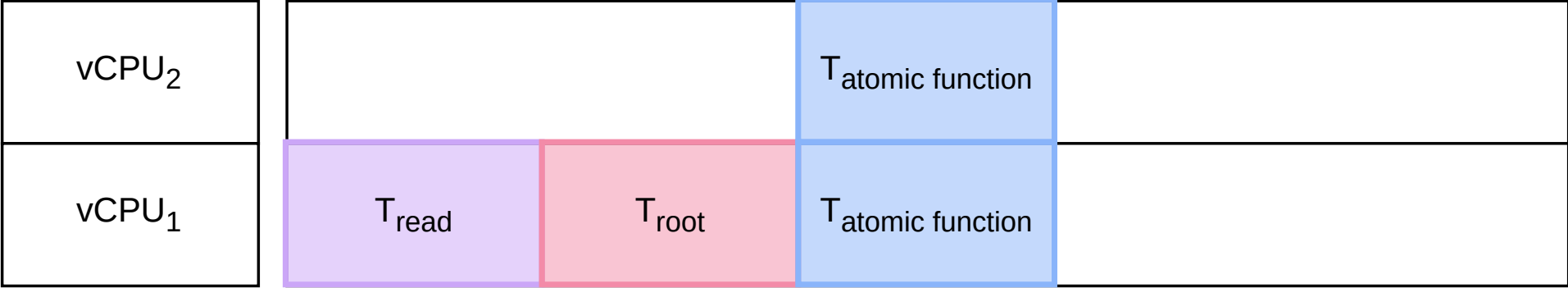
Atomi függvények külön szálon, kompozit függvények Step Functions Distributed Map segítségével külön Lambda példányban

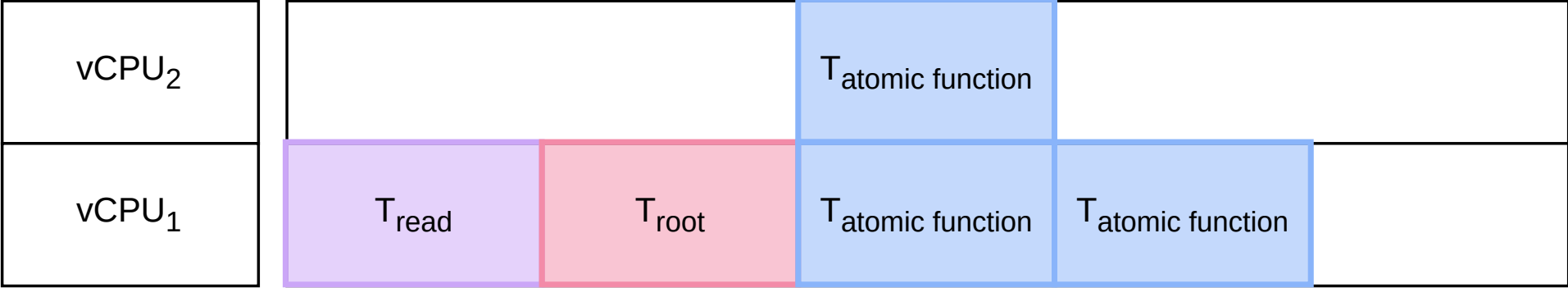
- parallel segédfüggvény: egy típuskulcs összes elemét külön szálon dolgozza fel
- Lambda-ból Lambda hívása nem ajánlott
- Step Functions natívan kezeli az állapotátmeneteket
- Distributed Map: S3 könyvtár alapján dinamikusan indítja a párhuzamos Lambda példányokat

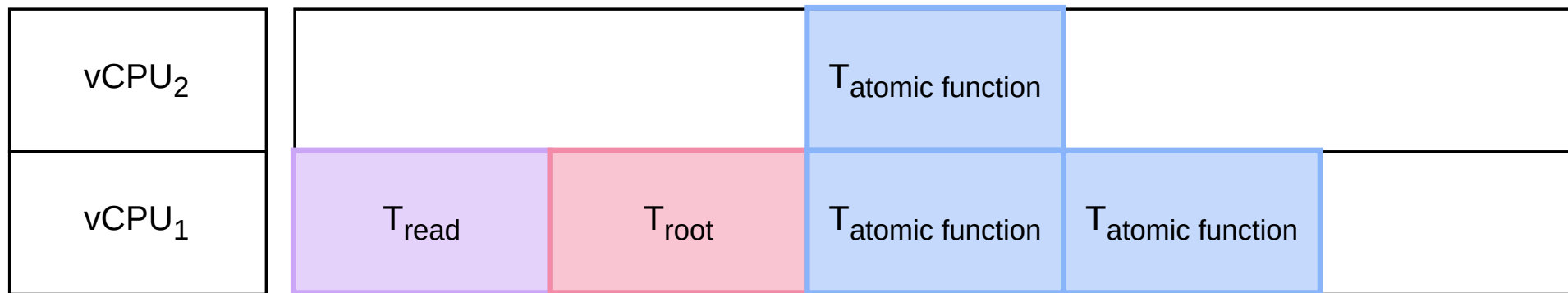




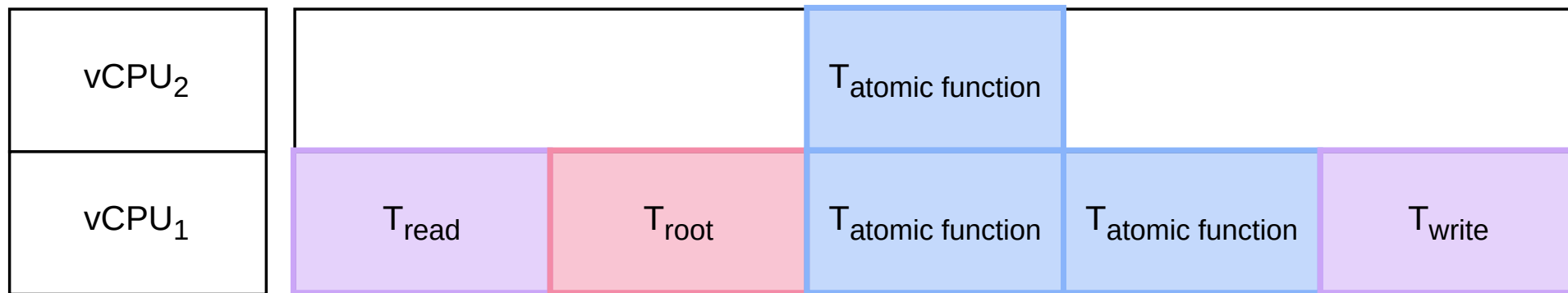






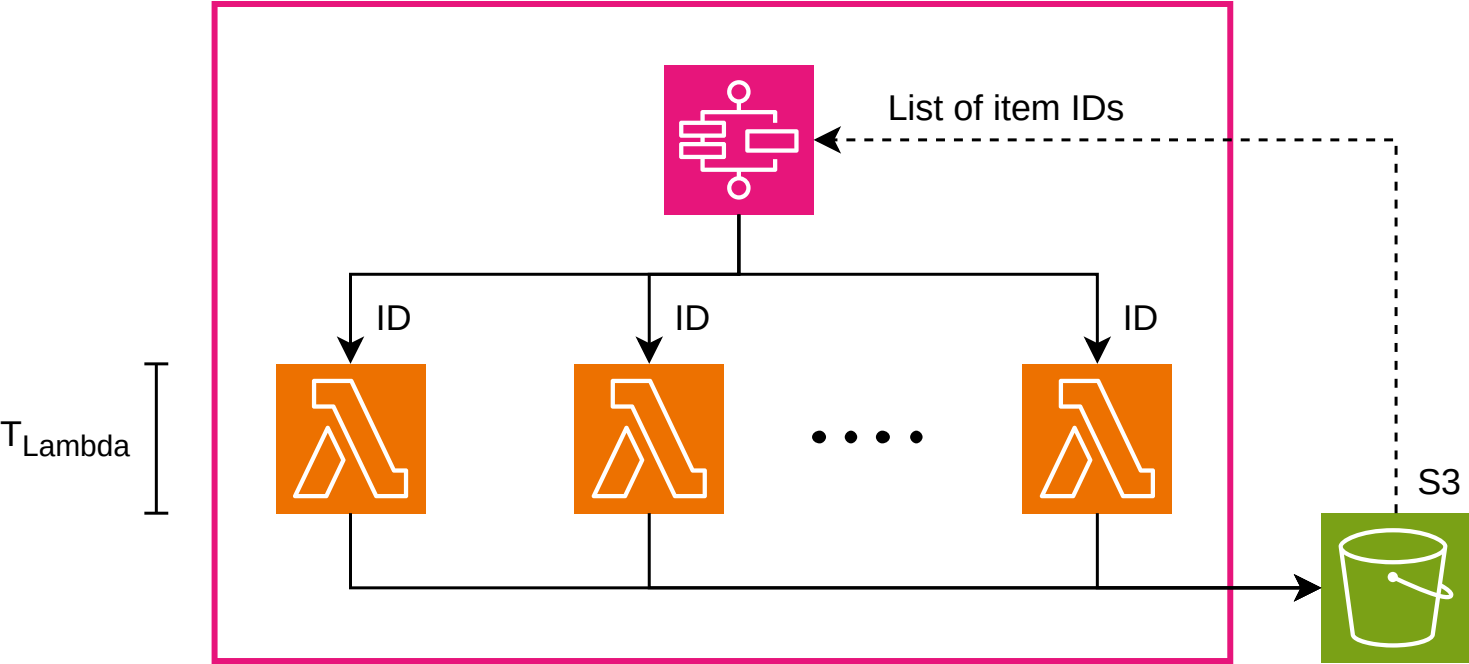


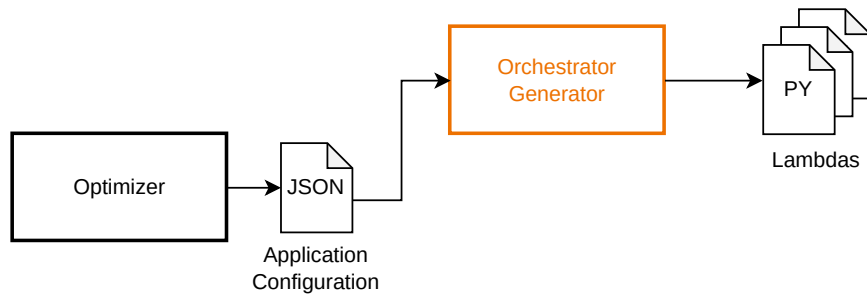
$$\left\lceil \frac{\text{number of inputs}}{\text{number of CPU cores}} \right\rceil * T_{\text{atomic function}}$$

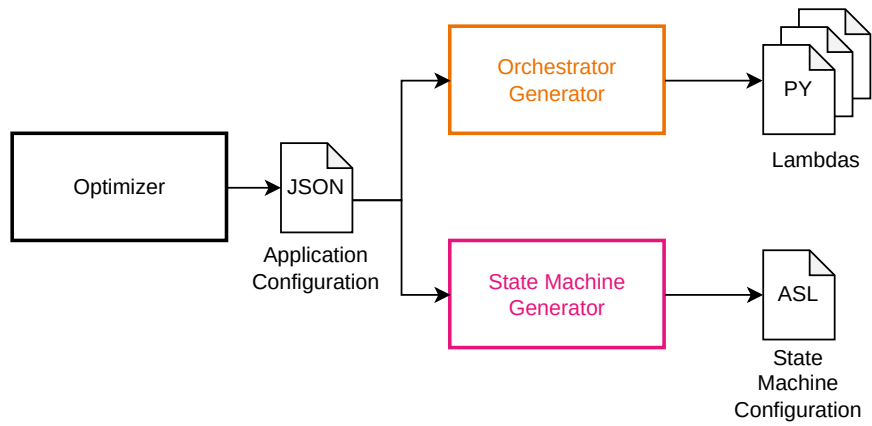


$$\left\lceil \frac{\text{number of inputs}}{\text{number of CPU cores}} \right\rceil * T_{\text{atomic function}}$$

Step Functions Distributed Map



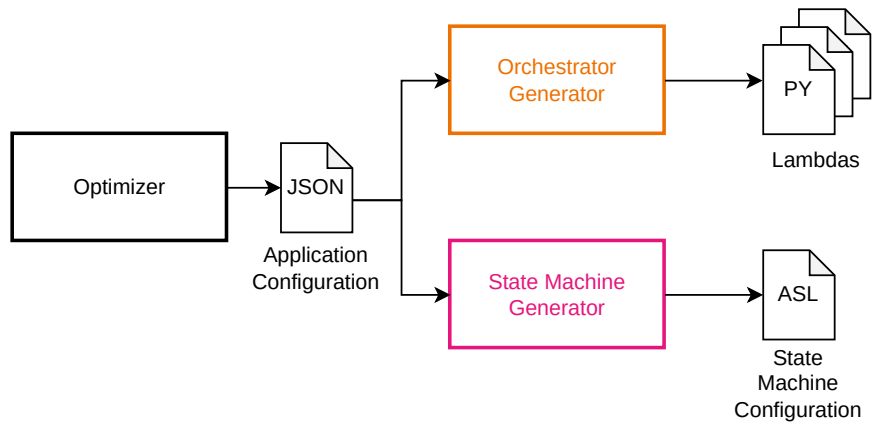


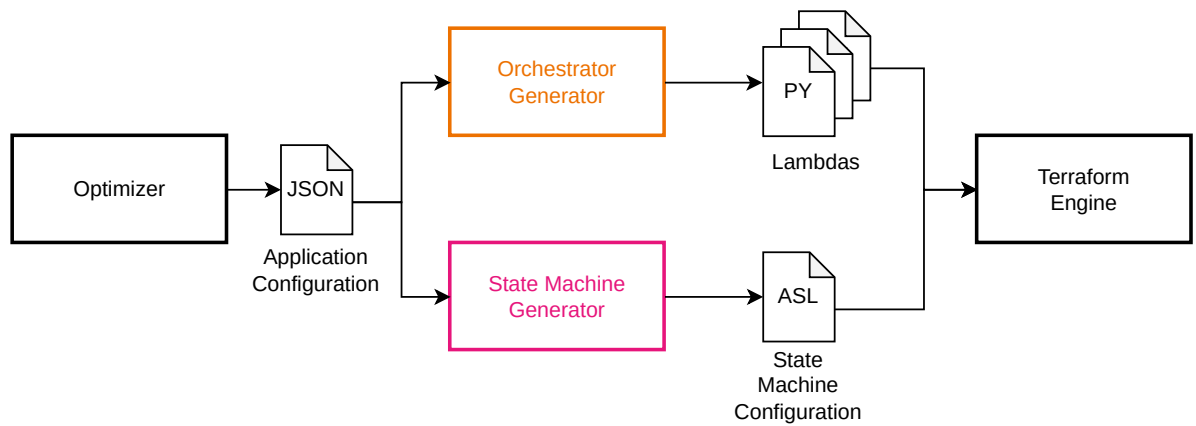


STEP FUNCTIONS

Az ASL állapotgép definíció automatikusan generálódik, a gyökér Task, a többi Distributed Map

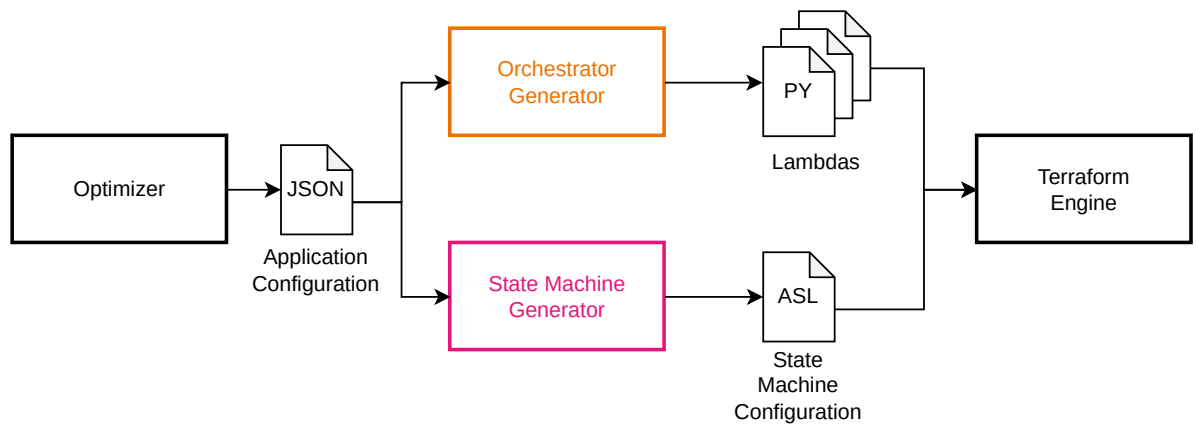
- Az első kompozit egyetlen Task állapot
- A gyökér Lambda mindig egy példányban fut
- A többi kompozit Distributed Map: S3 könyvtárból olvassa az azonosítókat
- ASL definíció is az optimalizáló kimenetéből generálódik

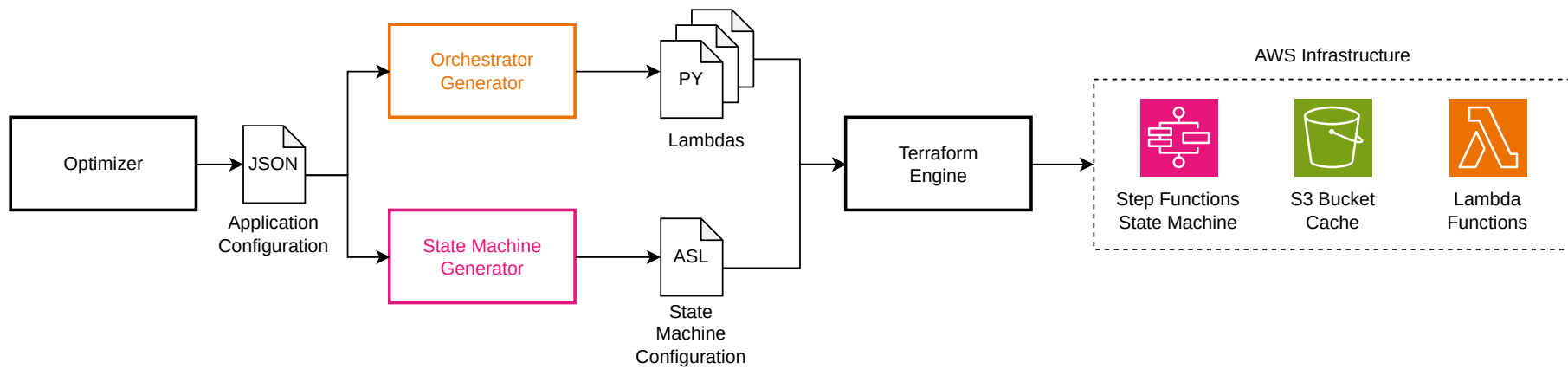


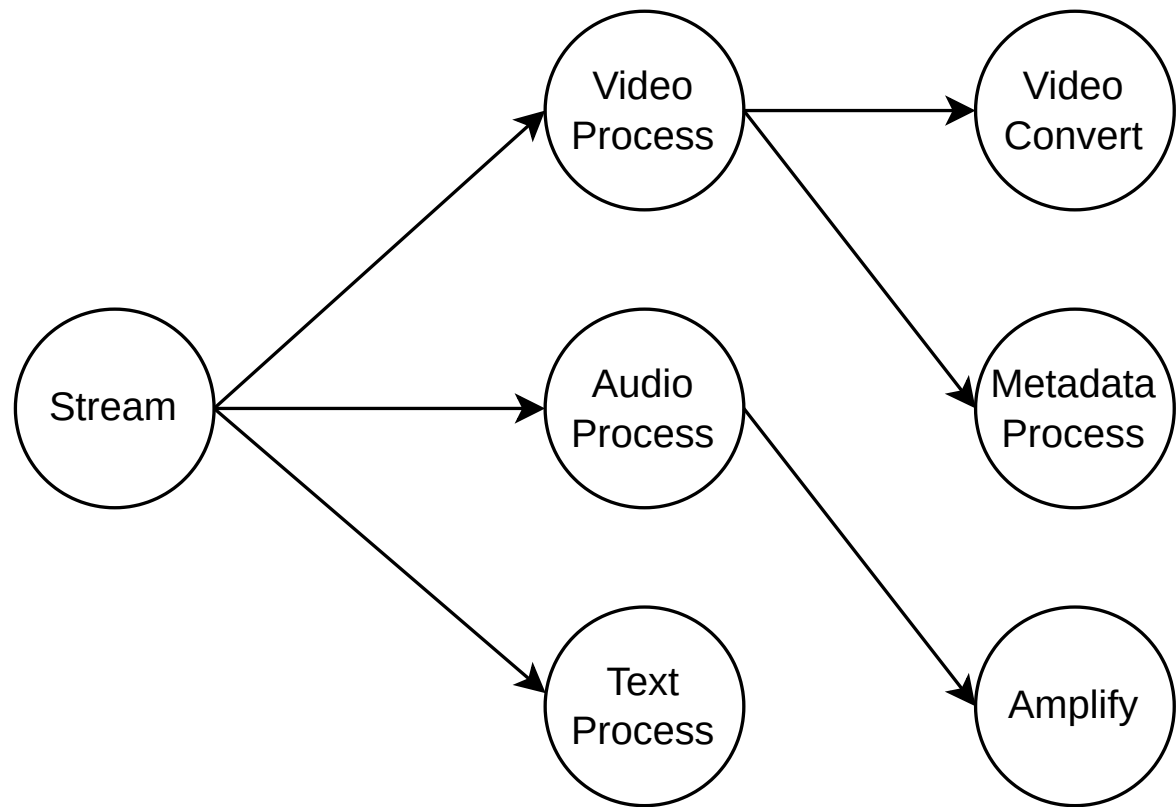


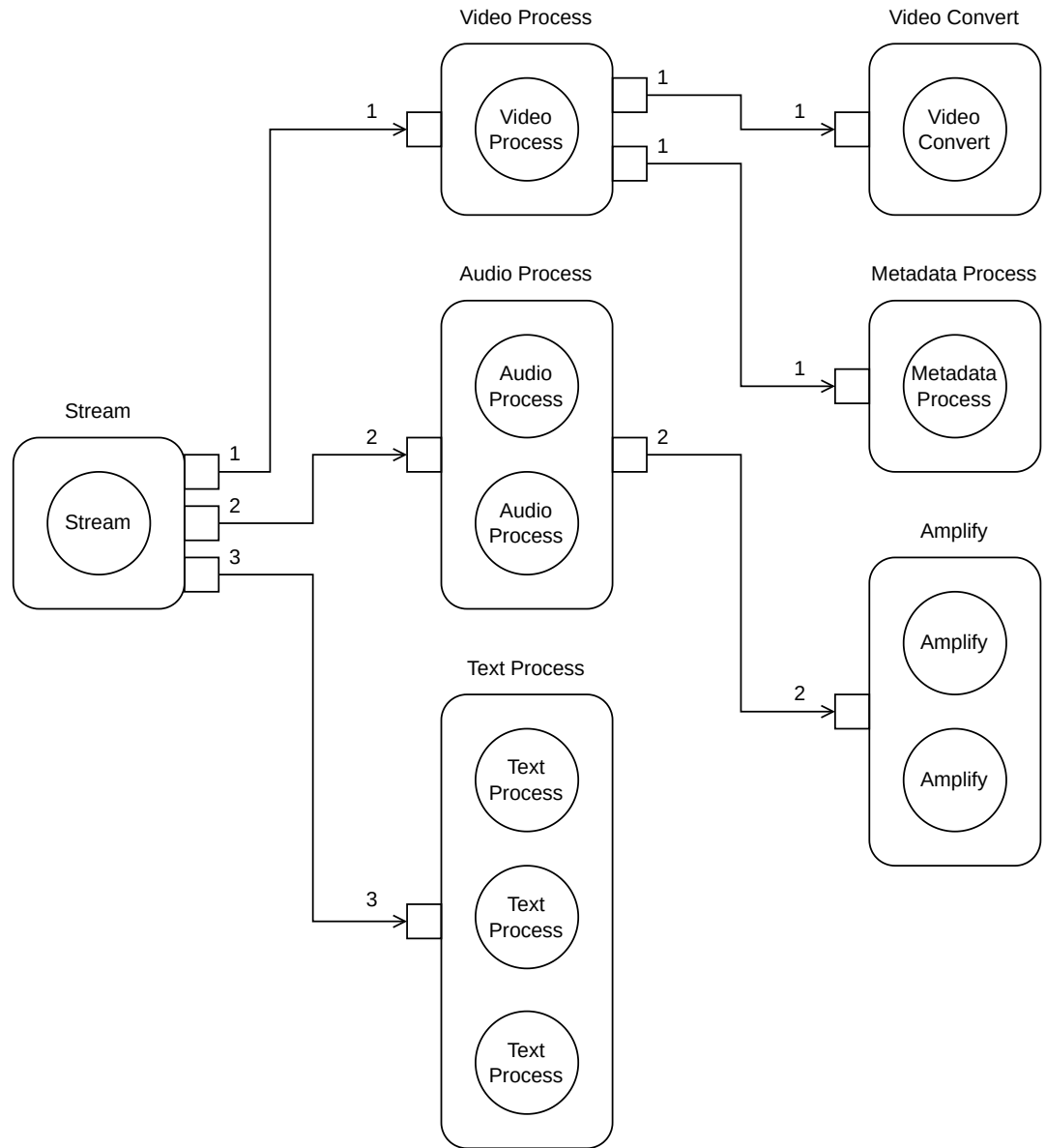
A teljes AWS infrastruktúra automatikusan deployolható: Lambda függvények, S3 bucket, Step Functions állapotgép

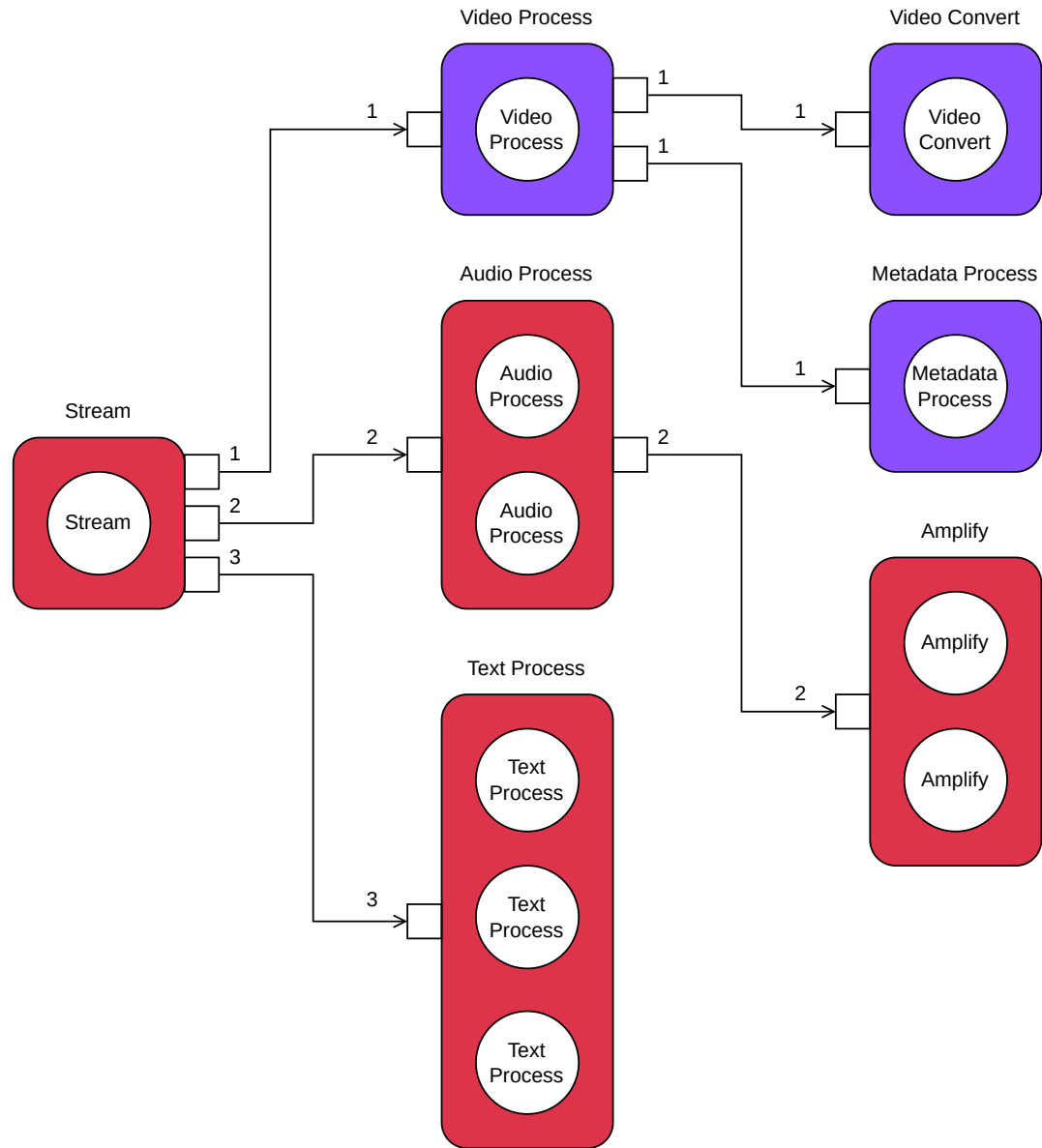
- Lambda forráskód feltöltése
- Tartalmazza az atomi függvények forráskódját és a generált orkesztrátort
- S3 bucket létrehozása cache-nek
- Generált ASL fájl feltöltése állapotgépként
- Bash script orkesztrálja a teljes pipeline-t a bemenettől a futásra kész alkalmazásig











EREDMÉNYEK

Kevesebb, mint 20%-ra esett vissza a végrehajtás ideje

- Minden atomi függvény külön Lambda egységbe: 16,07s
- Két kompozit Lambda függvénybe csoportosítás: 2,93s
- Drága állapotkezelés

Fejlesztési lehetőségek

- S3 helyett gyorsabb cache (Redis)
- Az állapotgép és függvények monitorozása
- Az így kinyert adatok az optimalizálóba táplálása
- Teljesen önálló, magát optimalizáló alkalmazás